BECKETT

DESIGN PROPOSAL · OPS-151 · --project beckett

Run of Show
a workflow engine the concierge conducts

A proposal to retire Plane as Beckett's execution backend and replace its one fixed ticket lifecycle with a per-task, concierge-authored workflow: stages, iteration budgets, parallel fan-out and gates composed at filing time — not compiled into the dispatcher.

Author
Beckett — worker seat
Claude Fable 5 · high

Requested by
ro (owner)
Discord · OPS-151

Date
12 July 2026
rev 1

Status
Proposal — no code
grounded in v3.2 source

IN ONE PAGE

TL;DR

Every piece of work Beckett takes on today — a copy tweak, a concurrency fix, a self-modification — marches through the same compiled-in pipeline: `todo → in_progress → in_review → done`, one implement seat, at most one review seat, three rework cycles, one live worker per ticket. That shape lives half in Plane (a remote SaaS ticket board we poll every few seconds) and half in a 3,200-line dispatcher switch statement. Changing it means changing Beckett's source and redeploying: adding *one* design stage cost us a new board, two new enum values, and surgery in five modules (OPS-111).

This document proposes making the workflow itself **data**: a small per-task DAG — the *run of show* — that the concierge writes when it files work. Stages, iteration caps, parallel fan-out, join rules and human gates become fields in a spec, interpreted by a generic engine that keeps everything already proven: worktrees and the task/branch tree, per-stage casting, review gates, the journal, the courier, steering. Plane is first demoted to a read-only mirror, then retired.

HOW TO READ THIS

Everything in §1 is fact, verified against the working tree at commit `6fac13d` with `file:line` receipts. Everything from §3 on is proposal. The two legacy lifecycles reappear in Appendix A as flow specs, which is the backward-compatibility argument in one page: if the engine can run *those two specs* identically, cutover changes nothing until we ask it to.

WHAT EXISTS · VERIFIED AGAINST SOURCE

1

Ground truth — how Plane runs Beckett today

Beckett's execution backend is a self-hosted **Plane** instance (`plane.0xbeckett.me`). The concierge files work as Plane issues; a poller diffs the board into events; a dispatcher turns events into worker processes in git worktrees. One paragraph of architecture, five subsystems of detail.

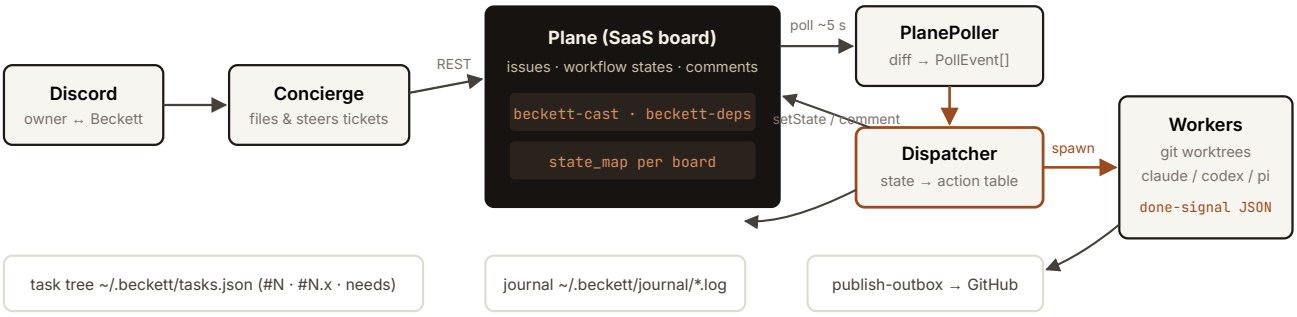

Fig. 1 — the v3 loop. The concierge writes to Plane; the poller diffs Plane into events; the dispatcher spawns workers and writes state back. All durable local artifacts (task tree, journal, outboxes) already live outside Plane.

1.1 The state machine and its enum

`TicketState` is a fixed eight-value union `src/plane/types.ts:26` — `backlog` , `todo` , `design` , `design_review` , `in_progress` , `in_review` , `done` , `cancelled` — with `done` / `cancelled` terminal `types.ts:39`. Each Plane board carries a `state_map` renaming these onto its columns `src/config.ts:129`. The dispatcher consumes poll events through one hardcoded dispatch table `src/dispatch/dispatcher.ts:1003-1062`:

EVENT / STATE ENTERED	GUARD	ACTION
`design`	INT board only, no live worker	spawn stage `design`
`in_progress`	no live worker	spawn stage `implement`
`in_review`	no publish hold, no live worker	spawn stage `review`
`done`	—	reap · drain orphaned steers · `promoteDependents()`
`todo` / `backlog` / `design_review`	—	park: abort+reap, commit WIP, keep worktree
`cancelled`	—	abort · reap · dispose
`comment_added`	live worker, author ≠ Beckett	`handle.nudge(body)` — steering

Stage names are string literals switched on in three places: the spawn table above, `castFor()` `dispatcher.ts:1998-2005` and the worker-done router `dispatcher.ts:2152-2161` (`design` , `design_check` , `implement` , `review`). A new stage is therefore a source change in at least three switch statements plus the enum, the poller's recovery list, and every board's `state_map` .

1.2 Casting, effort and the review gate

Casting is per-stage: a ticket description carries a fenced `beckett-cast` JSON block mapping stage name → `HarnessSpec` (`{harness, model, effort, reviewTier}`) `src/plane/cast.ts:29-59`. The `Casting` record is deliberately open-ended — arbitrary stage names already type-check `src/plane/types.ts:71-75` — but only the four hardcoded stage strings are ever spawned. Two consequential rules:

- **The review gate is inferred from a model knob.** `reviewTierFor()` `dispatcher.ts:2475-2479`: implement effort `low|medium` → `self` (one pass, straight to done); `high|xhigh|unset` → `fresh` (spawn an adversarial reviewer). An explicit `reviewTier` field overrides — a patch bolted on precisely because effort was overloaded.
- **Effort also sets the envelope** — turn caps and wall-clock via `ENVELOPE_BY_EFFORT` `src/dispatch/spawn.ts:217-222` (low 15 turns/10 min … xhigh 100 turns/60 min).

Named cast presets live in `~/.beckett/presets.json`, hot-reloaded on every filing `src/plane/presets.ts:76-112` — a precedent this proposal leans on: *per-task behaviour as user-editable data worked*.

1.3 Iteration policy: four global constants

LOOP	CAP	ON EXHAUSTION
review fail → re-implement (“rework”)	`MAX_REWORK_CYCLES = 3` `dispatcher.ts:199`	comment, park in `in_review` for a human
implement crash/timeout retry	`MAX_IMPLEMENT_RETRIES = 3` `dispatcher.ts:209`	publish WIP, move to `todo`
reviewer infra failure	`MAX_REVIEW_INFRA_RETRIES = 1` `dispatcher.ts:212`	park in `in_review`
design ↔ completeness check	`MAX_DESIGN_CYCLES = 2` `dispatcher.ts:202`	escalate to `design_review` with ⚠ note

These are compile-time constants. No ticket can ask for one shot, or six; the concierge’s only lever over iteration is not filing the work.

1.4 One worker per ticket, worktrees, done-signals

The dispatcher enforces **at most one live worker per ticket** (the `workers` map, staffing dedup and FIFO pending queue `dispatcher.ts:450, 1313-1379`) under a global `max_workers` cap. A spawn allocates a persistent worktree at `<repo>/.beckett/worktrees/<id>` on branch `beckett/task-N[.x]`, installs the scope-guard and the done-signal schema, captures the review base SHA on first implement `dispatcher.ts:1573-1586`, and periodically checkpoint-commits live work (OPS-125) `dispatcher.ts:640-709`. Workers finish by emitting a structured done-signal `{status: complete|blocked|partial, summary, filesChanged, checksRun, blockedReason}` `spawn.ts:198-209` — the single edge the state machine branches on.

1.5 Steering, journal, courier, task tree

- **Steering.** A human comment on an in-flight ticket becomes a live `nudge` (receipts: delivered / queued / will-restart / dropped); with no live worker it is buffered in `pendingSteers`, persisted, and folded into the next worker's brief `dispatcher.ts:1070–1158, 1609–1635`. Plane comments are the transport; Discord is where the human actually typed.
- **Journal.** An append-only per-ticket log under `~/.beckett/journal/<ticket>.log`, fed fire-and-forget from worker events, read on demand via `beckett journal <ticket> --tail N` `src/progress/journal.ts`. Already fully local; already richer than Plane's comment trail.
- **Courier / publish.** Finishing a ticket publishes to GitHub via a durable JSONL outbox with 1 m/5 m/30 m retries; `dispatcher.courier(id)` hands exclusive publish ownership to a human `src/dispatch/publish-outbox.ts, dispatcher.ts:831–839`. Plane state writes themselves go through a second durable outbox (`advance-outbox`) because remote writes fail often enough to need one `src/dispatch/advance-outbox.ts`.
- **Task tree.** The user-facing `#N` / `#N.x` tree with `needs` dependencies lives in `~/.beckett/tasks.json` — a local, locked, versioned registry that already mirrors every Plane state into a branch status (`backlog→waiting, … in_review→review`) `src/task/store.ts:141–152`. Cross-ticket DAG promotion (`blockedBy` → promote on done) is dispatcher logic, not Plane's `dispatcher.ts:2889–2926`.

THE LOAD-BEARING OBSERVATION

Plane contributes exactly four things: issue storage, a state column per ticket, comments-as-transport, and a human-viewable board. Everything Beckett actually *runs on* — worktrees, casting, envelopes, done-signals, steering buffers, journal, outboxes, the #N tree, DAG promotion — is already local, already durable, and already ours. The migration in §6 is small precisely because of this.

WHY CHANGE · SIX CONCRETE FAILURES

2

The problem — one lifecycle, worn as a straitjacket

Nothing in §1 is broken code. The machine is careful, durable and observable. What it cannot do is *vary*: the shape of work is compiled in, so every task — whatever its nature — is forced through the same pipe, and every new shape is a Beckett source change plus a deploy.

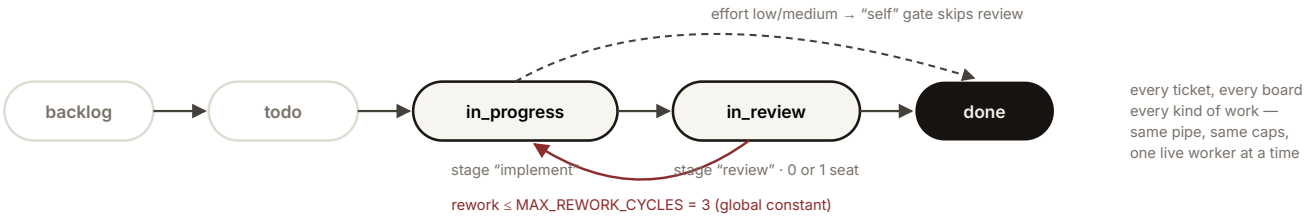

Fig. 2 — the compiled-in lifecycle. The only per-task levers are which model sits in each of two seats and an effort knob that doubles, implicitly, as the review-gate switch.

2.1 Case study: adding one stage took a board, an enum, and five modules

OPS-111 wanted exactly one new thing: a *design* stage with a human approval gate before implementation. Because the lifecycle is compiled in, the delivered change (OPS-111 → build) required: a **new Plane board** (INT) so OPS wouldn't grow columns; **two new enum values** (`design` , `design_review`) in `TicketState` ; board-guarded arms in the dispatcher switch; a new `design_check` pseudo-stage with its own hardcoded cast (Haiku, low) and its own constant (`MAX_DESIGN_CYCLES`); poller `prime()` changes so restarts re-staff `design` but not `design_review` ; and Plane-side state provisioning with the right column groups and colours `docs/design/int-flow.md · dispatcher.ts:2180–2239 · poll.ts:180`. That is the cost of *one* stage, designed by the book. The next shape — say, a research stage, or a second reviewer — costs the same again. The lifecycle has a marginal cost per variant that data should have made ~zero.

2.2 Iteration budgets are global, not per task

A README tweak and a subtle concurrency fix both get exactly three rework cycles, three crash retries, two design bounces (§1.3). The concierge — the seat with actual context about stakes, urgency and budget — cannot say "one pass, no retries, this is trivial" or "keep iterating up to six rounds against the eval, it's worth it." When a cap is wrong the options are: edit a constant in `dispatcher.ts` and redeploy, or babysit manually via state flips.

2.3 No parallelism inside a task

One live worker per ticket is structural `dispatcher.ts:450`. Things ro has wanted that this forbids outright:

- **Bake-offs** — three candidate implementations from different casts, one judge picks a winner. Today that is three hand-filed tickets, no shared base management, and the judging is ro reading diffs.

- **Panel review** — correctness, security and taste reviewers running *concurrently* on one diff. Today: one reviewer seat, serial by construction.
- **Research fan-out** — N scouts reading different subsystems before a synthesis stage. Today this only exists *inside* a single worker's subagents, invisible to the engine, unsteerable per-branch, and un-cast (the worker picks its own subagents; the concierge can't put Fable on one scout and Haiku on the rest).

The only parallelism the engine knows is *between tickets* via `beckett plan --needs` — the right tool for genuinely separate deliverables, wrong for branches of one task that share a worktree lineage and must join.

2.4 The workflow is inferred from a model knob

Whether a ticket gets reviewed at all is a *side effect of effort* (§1.2) — and an *omitted* effort silently buys the expensive fresh-review tier. `reviewTier` exists only to patch this overload. The shape of the pipeline should be a first-class, visible declaration, not something reverse-engineered from how hard the implement seat is thinking. (The presets doc already flags this as a foot-gun: "presets must always name an explicit effort" `docs/design/cast-presets.md §1`.)

2.5 State lives in someone else's database, structured data in a Markdown field

Every stage transition is a remote HTTP write to Plane — unreliable enough that a durable `advance-outbox` exists solely to retry them `src/dispatch/advance-outbox.ts` — and every reaction is gated on a ~5 s poll diff. Meanwhile the actually-structured task data (cast, dependencies, project, branch ref, start state, origin channel) is smuggled through Plane's description field as five fenced code blocks and an HTML comment `cast.ts:29-42`, because the board has no schema for any of it. The VID board bends video production onto the same six coarse states, and the client folds any unrecognised "started" column back to `in_progress` to survive human-added columns `client.ts:574`. These are the classic scars of a tool being used as a database it isn't.

2.6 The concierge can't compose; it can only pick

The concierge's authoring surface today is: title, body, criteria, board, entry state, and which models sit in the fixed seats (`--cast` / `--preset`). It cannot add a stage, split a stage, gate a stage, cap a loop, or fan anything out — the exact dimensions ro keeps asking to vary per task. Every one of those requests currently becomes an OPS ticket against Beckett's own source.

> **THE REQUIREMENT, STATED ONCE**
>
> The concierge must be able to **dictate, per task, how many stages there are, what runs in each, how many iterations each loop gets, and how much parallelism to spend** — at filing time, as data, without touching Beckett's source or Plane's schema. Everything in §3–§5 is the smallest design we can find that delivers exactly that and nothing more.

THE PROPOSAL · WORKFLOW AS DATA

3

The proposal — a per-task flow spec

Replace the compiled-in lifecycle with a small, validated document — the **flow spec**, or *run of show* — attached to every task at filing time. The dispatcher's hardcoded state→action table becomes a generic interpreter over exactly **four node kinds**. Everything below the node boundary (worktrees, casting, envelopes, done-signals, journal, steering, courier) is reused unchanged.

3.1 Design principles

- **Small algebra, closed set.** Four node kinds: `worker`, `gate`, `fanout`, `join`. No expressions, no conditionals beyond pass/fail edges, no user-defined node types. Anything fancier must arrive as a fifth kind with its own design doc — that is the fence against the workflow-engine tarpit (§7.1).
- **Every loop is bounded by construction.** A spec that could run forever does not validate. Caps are per-node fields, defaulted, not global constants.
- **Today's behaviour is two specs.** The OPS and INT lifecycles must be expressible byte-for-byte (Appendix A), so cutover is a no-op until the concierge starts writing new shapes.
- **The concierge composes; the engine conducts.** Authoring is data + presets (§5); execution, recovery and budget enforcement are the engine's (§4). The worker never sees the flow — it still gets a brief, a worktree, and a done-signal schema.

3.2 The data model

```
// All of this validates with zod at filing time, like castings do today. interface FlowSpec { version: 1;
```

The spec travels in a fenced `beckett-flow` block exactly where `beckett-cast` lives today (and, after migration, as a typed field in the local task record — §6). Casting stays a separate concern: the flow names *seats*; the cast fills them. A flow with no `beckett-flow` block is compiled from the cast exactly as today (effort → gate), so existing filings never break.

3.3 Validation — what the linter refuses at filing time

- Unknown edge targets; unreachable nodes; an `entry` that isn't a node; fanout arms whose edges escape their join.
- **Unbounded cycles:** every cycle in the graph must pass through a node whose `maxVisits` / `maxFails` is finite (all default finite; setting one to `Infinity` is simply not representable).
- Budget sanity: `maxConcurrent` ≤ global `max_workers`; a Fable seat anywhere in the spec triggers the existing confirm-before-cast handshake — flow authoring does not bypass the cast doctrine.
- Cast validation per seat via the existing `validateCasting` (blocked models, malformed efforts) `presets.ts:76-112`.

3.4 What today's shapes look like (proof of containment)

The default reviewed lifecycle, as a spec — this is *all* of it:

```
{ "version": 1, "entry": "implement", "nodes": { "implement": { "kind":"worker", "cast":{"harness":"pi","e
```

One-pass work is the same spec minus the gate (`onPass:"done"`). The INT design flow adds two nodes in front — a `design` worker and a `human` gate — instead of two enum values, a board, and five modules. All three appear in full in Appendix A.

3.5 And what today cannot do at all

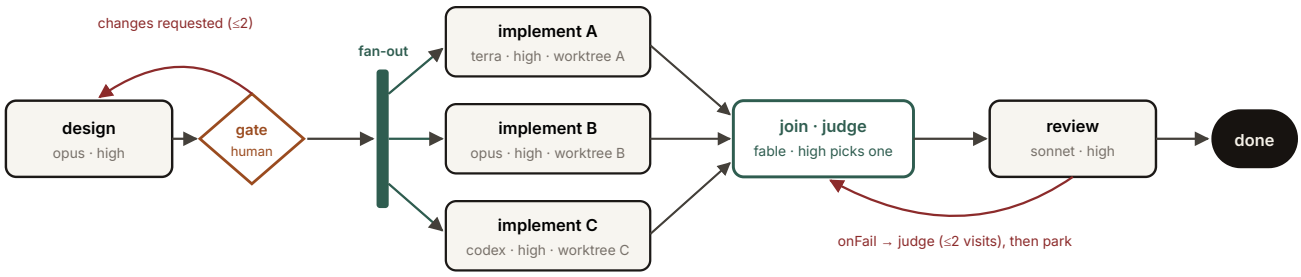

Fig. 3 — a bake-off run of show. Design → human gate → three isolated candidate implementations → a judge join that picks (or synthesises) a winner → fresh review → done. Six nodes of data. Under the current engine this is three hand-filed tickets, a human doing the judging, and no joint lineage.

RUNTIME · THE STAGE MANAGER

4

Execution semantics

The engine — call it the **stage manager** — is a per-task interpreter with one job: hold a durable `FlowRun` record, advance cursors along edges when done-signals arrive, and enforce budgets. It replaces the dispatcher's state-switch; it deliberately does *not* replace the spawn path, the drivers, or any durability machinery.

4.1 The run record

```
interface FlowRun { taskRef: string; // "#42.1" — the existing tree ref stays the primary key spec: FlowSp
```

Runs are **event-sourced**: every transition (`node_entered` , `signal_received` , `edge_taken` , `parked` , `budget_hit` …) is appended to `~/.beckett/runs/<ref>.jsonl` and the record above is just the fold. This is the same durability idiom the codebase already trusts — the journal, the advance-outbox and the publish-outbox are all append-only JSONL `journal.ts · advance-outbox.ts:49-104` — promoted to being the source of truth instead of a shadow of Plane's.

4.2 How a stage runs — nothing new below the node

Entering a `worker` node calls today's spawn path verbatim `dispatcher.ts:1479-1729 · spawn.ts:491`: allocate/reuse the task worktree, install scope-guard + done-signal schema, build the brief (ticket body + node `brief` + drained steers), apply the effort envelope, register checkpoint commits. The node completes when the done-signal arrives: `complete` takes `onPass` ; `blocked` / `partial` /process-error takes `onFail` (after same-visit `retries` for infra failures, preserving today's crash/auth/rate-limit ladder `dispatcher.ts:1844-1930, 2356-2442`). The LIVE/PARKED distinction survives intact: `worker` and model-`gate` nodes are LIVE; `human` gates and `park` are PARKED — no process, zero tokens, restart-inert, exactly the property `design_review` proved out `poll.ts:242-293`.

4.3 How loops terminate — always, by construction

- **Per-node:** entering a node increments `visits[node]` ; exceeding `maxVisits` (or a gate's `maxFails`) diverts the edge to `park` with a journal note and a concierge ping — today's "three strikes then leave in `in_review` for a human" `dispatcher.ts:2756-2783`, generalised and made per-task data.
- **Per-run:** the `budget` ceilings (spend, wall-clock) are checked before every spawn; a breach parks the run with `budget_hit` . A parked run never burns another token until a human (or the concierge, with authority) resumes it.
- **Statically:** the linter already refused any cycle not cut by a finite counter (§3.3), so the dynamic checks are enforcement, not the proof.

4.4 How parallel branches run and join

A `fanout` node forks the cursor: each arm gets its own worktree and branch — `beckett/task-42.1/arm-2` — via the same `createWorktree` machinery, from the same base SHA

(captured exactly as the review base is today `dispatcher.ts:1573-1586`). Arms count against `budget.maxConcurrent` and the global `max_workers` cap; excess arms queue in the existing FIFO. Join strategies:

STRATEGY	SEMANTICS	REUSES
`all-merge`	Wait for every arm's `complete`; merge arm branches into the task branch in arm order; conflict → `onFail`.	`mergeBranchesIntoWorktree()` — already merges dependency branches today `worktree.ts`
`first`	First `complete` wins; engine aborts remaining arms (`handle.abort`), reaps their worktrees.	the cancel path `dispatcher.ts:996`
`quorum`	For verdict-shaped arms (panel review): pass when `quorumK` arms pass; fail when impossible.	done-signal statuses as votes
`judge`	Spawn the judge cast with every arm's diff + summary as its brief; its done-signal names the winning arm (merged) or requests a synthesis pass.	the review-diff builder `dispatcher.ts:1607-1623`

An arm that fails takes the *arm's* failure into the join: `all-merge` fails fast, `first` ignores it unless all arms fail, `quorum` counts it against, `judge` receives it as context. A cancelled run aborts all cursors — same semantics as today's single worker, mapped over the set.

4.5 Steering and observability

Steering stops being "a Plane comment we poll for" and becomes a first-class verb: `beckett task nudge <ref> [--node id] "text"`, called by the concierge when a human replies on the task's origin channel (the channel linkage already exists — `originChannel` is stamped on every filing `types.ts:114-117`). Un-targeted nudges go to all live cursors; targeted ones to a node's worker or its buffer. Delivery semantics — receipts, buffering when nothing is live, folding into the next brief, unapplied-nudge drain — are today's, unchanged `dispatcher.ts:1070-1158`. The journal gains node context per line; `beckett status` renders each run's DAG with cursor positions, visit counts and spend — which is *more* visibility than the Plane board offers now, not less.

4.6 Crash recovery

On boot the engine replays each run's event log (no Plane `prime()` round-trip): PARKED cursors stay parked; LIVE cursors re-staff from their checkpointed worktrees using the existing resumables/session-resume bridge `dispatcher.ts:583-627`. The one-writer-at-a-time discipline the ledger provides today extends per-cursor. This *strengthens* recovery: today's restart re-derives intent from Plane state; tomorrow's replays exactly what the engine was doing.

AUTHORING · WHAT THE CONCIERGE WRITES

5

The authoring surface

Filing keeps its shape: `beckett task file` with title, body, criteria, project, channel. The flow arrives three ways, cheapest first — and the cheapest way is *not writing one*.

5.1 Defaults — most filings never mention a flow

No `--flow` → the engine compiles the spec from the cast exactly as today: effort `low|medium` → one-pass; `high|xhigh` → implement + review gate; `reviewTier` respected. Zero behaviour change, zero new typing for the everyday ticket. (One deliberate improvement: the compiled spec is *printed back* on filing — the gate becomes visible instead of inferred silently.)

5.2 Flow presets — named shapes in `~/.beckett/flows.json`

The OPS-110 pattern, re-applied: hot-reloaded, user-editable, validated on load, seeded with the vocabulary we already know we want:

PRESET	SHAPE	REPLACES / ENABLES
`one-pass`	implement → done	today's self tier, said out loud
`reviewed`	implement → review ⟲≤3 → done	today's fresh tier
`intensive`	design → human gate ⟲≤2 → implement → review → done	the whole INT board, as 4 nodes of data
`bake-off`	fanout(n candidates) → judge → review → done	Fig. 3 — impossible today
`panel-review`	implement → fanout(correctness, security, taste) → quorum 2/3 ⟲ → done	parallel reviewers — impossible today
`iterate-until`	implement ⟲≤N against a model-gate rubric (eval/tests) → done	per-task iteration budgets — the constants, freed

Invocation: `--flow-preset bake-off`, with per-node overrides via `--flow-set judge.cast.model=claude-fable-5` or a partial-JSON merge — same precedence story as cast presets (explicit beats preset, per stage) `presets.ts:119-142`.

5.3 A shorthand for composing fresh — the one-liner grammar

For shapes worth composing on the spot, a tiny expression syntax compiled to a `FlowSpec` (and echoed back as the expanded JSON before filing):

```
stage(cast) worker node design(opus:high) ?human / ?cast gate ?human · ?haiku:low(rubric=criteria) a → b
```

```
# what I'd actually type when ro asks for a bake-off with a design gate: beckett task file --title "…" --p
```

The grammar is sugar only — it must compile to the §3.2 model and validate identically. If it ever wants a feature the model lacks, the model wins and the feature waits for a design doc.

5.4 What stays exactly as it is

- `beckett plan` **and the task tree.** Inter-task DAGs (`--needs` , `#N.x` , `blockedBy` promotion) are the right tool for separate deliverables and keep working unchanged; a plan node may now also carry a `flow` . Flows are *intra*-task; plans are *inter*-task. The two compose and do not compete.
- **Casting doctrine.** Presets, the roster, effort envelopes, Fable confirm-first — all untouched. A flow references seats; the cast doctrine still decides who may sit in them.
- **The worker's world.** Brief in, worktree, scope-guard, steering nudges, done-signal out. A worker cannot tell whether it is running under the old dispatcher or the stage manager — that is the compatibility invariant the whole migration hangs on.